

Exp No: 28

COUNTING SEMAPHORE IN SYSTEMC

AIM:

To implement Counting Semaphore allowing multiple resource access.

SOFTWARE REQUIRED:

- Ubuntu (or WSL Ubuntu)
- g++
- CMake
- SystemC

PROGRAM:

```
#include <systemc.h>
```

```
SC_MODULE(counting_semaphore) {  
    int sem;
```

```
    void task() {  
        while(true) {  
            wait(1, SC_SEC);  
  
            if(sem > 0) {  
                sem--;  
  
                cout << "Task Accessing Resource at "  
                    << sc_time_stamp()  
                    << " Remaining: " << sem << endl;  
  
                wait(2, SC_SEC);  
  
                sem++;  
            }  
        }  
    }  
}
```

```
SC_CTOR(counting_semaphore) {  
    sem = 2;  
  
    SC_THREAD(task);  
    SC_THREAD(task);  
    SC_THREAD(task);  
}
```

```
};
```

```
int sc_main(int argc, char* argv[]) {  
    counting_semaphore obj("Counting");
```

```
    sc_start(10, SC_SEC);
```

```
    return 0;
```

```
}
```

OUTPUT

The screenshot shows the EDA Playground web interface. The left sidebar has sections for "Languages & Libraries" (Testbench + Design, C++/SystemC, Libraries: SystemC 2.3.3), "Tools & Simulators" (C++, Compile Options: -DSC_INCLUDE_FX, C++14, Run Options), and "Examples". The main editor area has two tabs: "testbench.cpp" and "design.cpp". The "testbench.cpp" tab is active, showing a C++ program that uses SystemC to create a counting semaphore and start a task that accesses it 10 times. The "design.cpp" tab shows a placeholder for the user's design. The bottom log window shows the output of the simulation, which is "Done".

```
testbench.cpp
3 int sem;
4 void task() {
5     while(true) {
6         wait(1, SC_SEC);
7         if(sem > 0) {
8             sem--;
9             cout << "Task Accessing Resource at "
10                << sc_time_stamp()
11                << " Remaining: " << sem << endl;
12             wait(2, SC_SEC);
13             sem++;
14         }
15     }
16 }
17 SC_CTOR(counting_semaphore) {
18     sem = 2;
19     SC_THREAD(task);
20     SC_THREAD(task);
21     SC_THREAD(task);
22 }
23 };
24 int sc_main(int argc, char* argv[]) {
25     counting_semaphore obj("Counting");
26     sc_start(10, SC_SEC);
27     return 0;
28 }
```

```
design.cpp
1 // Code your design here.
2 // Uncomment the next line for SystemC modules.
3 // #include <systemc>
```

Log

```
Task Accessing Resource at 1 s Remaining: 0
Task Accessing Resource at 3 s Remaining: 0
Task Accessing Resource at 4 s Remaining: 0
Task Accessing Resource at 5 s Remaining: 0
Task Accessing Resource at 6 s Remaining: 0
Task Accessing Resource at 7 s Remaining: 0
Task Accessing Resource at 8 s Remaining: 0
Task Accessing Resource at 9 s Remaining: 0
Done
```

RESULT

Thus the Counting Semaphore was created using SystemC and verified.